



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Intermediate Code Generation for a Scientific / Financial Domain-Specific Language Compiler

Mini Project Report — Compiler Design

Submitted by:

M.Greeshma

192524195

CSA1401- Compiler Design]

SIMATS Engineering college

Table of Contents

1. Problem Statement.....	3
2. Objective	3
3. Requirements and Environment Used	4
4. Design / Proposed Solution.....	4
5. Algorithm / Pseudocode.....	5
6. Implementation / Source Code.....	7
7. Test Cases and Expected / Actual Results	13
8. Execution Screenshots / Output	15
9. Analysis and Discussion	17
10. Conclusion	18
11. Individual Contribution of Group Members	18
12. References.....	19

Note: page numbers above are approximate — exact pagination may shift slightly depending on the viewer/printer settings.

1. Problem Statement

A software development company is building a compiler front-/middle-end for a domain-specific language (DSL) intended for scientific and financial computing. Before such a compiler can emit optimized machine or bytecode instructions, it must first translate high-level arithmetic expressions into a machine-independent intermediate representation (IR). This project focuses specifically on that middle stage: intermediate code generation.

The concrete motivating expression, drawn from computational finance, is the calculation of a final investment value that combines simple interest with compound interest, less associated fees:

```
finalValue = ((principal * rate * time) / 100)
             + (principal * (1 + rate/100) ^ time)
             - fees
```

This expression is representative of the kind of nested, multi-operator arithmetic a scientific/financial DSL compiler must handle correctly and efficiently: it mixes four binary operators (+, -, *, /), a power operator (^), parentheses that override default precedence, integer and floating literals, and several independent variables.

The problem, therefore, is to design a pipeline that (a) parses such expressions into a structured syntax tree, (b) lowers the tree into a linear three-address code (TAC) sequence, (c) represents that sequence using the classical quadruple and triple data structures used in compiler textbooks, and (d) applies standard code-improving transformations to the intermediate representation before it is handed to a back-end code generator.

2. Objective

- Design a lexer and recursive-descent parser that converts an arithmetic assignment statement into an Abstract Syntax Tree (AST), correctly honouring operator precedence and associativity (including a right-associative power operator).
- Generate Three Address Code (TAC) from the AST using a standard post-order tree-walk with temporary-variable allocation.

- Convert the generated TAC into both the quadruple representation (op, arg1, arg2, result) and the triple representation (op, arg1, arg2, referenced positionally).
- Implement and apply at least three intermediate-code optimization techniques — constant folding, common sub-expression elimination (CSE), and dead code elimination (DCE) — as iterative, composable passes over the quadruple list.
- Quantitatively compare optimized versus non-optimized code in terms of instruction count and discuss the resulting gain in execution efficiency.
- Validate the pipeline on the finance expression above and on a second, purpose-built test program that is guaranteed to exercise every optimization rule so that the improvement can be observed unambiguously.

3. Requirements and Environment Used

3.1 Software / Tools

Component	Tool / Version used
Programming language	Python 3.11
Standard libraries	re (regular expressions), dataclasses, typing
Development environment	Linux (Ubuntu) sandboxed container, command-line execution
Report generation	docx (Node.js) for the Word document you are reading
Version control (recommended)	Git / GitHub

3.2 Functional Requirements

- Accept an assignment statement of the form `identifier = expression` (or a semicolon-separated sequence of such statements).
- Support operators `+`, `-`, `*`, `/`, `^`, parentheses, integer/decimal literals, and identifiers.
- Produce human-readable TAC, quadruple tables, and triple tables for both the optimized and unoptimized code.

- Produce a numeric comparison of instruction counts before and after optimization.

3.3 Non-Functional Requirements

- The implementation should be modular (separate lexer, parser, TAC generator, optimizer) so that each stage can be tested independently.
- Optimization passes must be safe: they must not change the meaning (the computed value) of the program.

4. Design / Proposed Solution

The system is implemented as a five-stage pipeline, mirroring the classical structure of a compiler front end / middle end:

- Lexer — converts the raw source string into a token stream (NUMBER, IDENT, operators, parentheses, '=', ';') using a single combined regular expression.
- Parser — a recursive-descent parser implementing the grammar in Section 5 builds an Abstract Syntax Tree (AST) out of Num, Var, BinOp and Assign node types.
- TAC Generator — a post-order traversal of the AST allocates a new temporary (t1, t2, ...) for every internal node and emits one three-address instruction per operation.
- IR Converters — the flat TAC/quadruple list is transformed into the triple representation by replacing temporary-variable references with positional pointers to the triple that produced them.
- Optimizer — three passes (constant folding, common sub-expression elimination, dead-code elimination) are applied repeatedly until no further reduction is possible (fixed-point iteration).

The following diagram summarizes the data flow:

Source text

| (Lexer: regex tokenizer)

v

Token stream

| (Recursive-descent Parser)

v

Abstract Syntax Tree (AST)

| (Post-order TAC Generator)

v

Three Address Code (TAC) / Quadruples <----> Triples

| (Optimizer: fold -> CSE -> DCE, repeat to fixed point)

v

Optimized TAC / Quadruples / Triples

|

v

(Ready for target/back-end code generation)

Design decisions worth noting:

- Quadruples store an explicit result field, which is what makes constant folding and CSE straightforward: later instructions are rewritten by substituting the constant / earlier temporary wherever the eliminated result was used.
- Triples derive the result implicitly from an instruction's position, so eliminating or reordering an instruction would require renumbering every reference — a classical argument for why quadruples are easier to optimize, while triples save the memory of a result field. Both are generated here to compare the two forms directly.
- Optimizations are expressed as pure functions taking a quadruple list and returning a new quadruple list plus a boolean 'changed' flag, so they can be composed and iterated safely (a standard fixed-point / work-list style optimizer).

5. Algorithm / Pseudocode

5.1 Grammar (used by the recursive-descent parser)

```
program -> assignment ( ';' assignment )*
```

```
assignment -> IDENT '=' expr
```

```
expr -> term ( ('+' | '-') term )*
```

```
term -> power ( ('*' | '/') power )*
```

```
power -> factor ( '^' power )? // right-associative
```

```
factor    -> NUMBER | IDENT | '(' expr ')' | '-' factor
```

5.2 TAC Generation (post-order tree walk)

```
function genExpr(node):
    if node is Num or Var: return node.value
    if node is BinOp(op, L, R):
        left = genExpr(L)
        right = genExpr(R)
        t = newTemp()
        emit( t = left op right )
        return t
```

```
function genAssign(target, expr):
    v = genExpr(expr)
    emit( target = v )
```

5.3 Constant Folding

```
for each quad (op, a1, a2, res) in order:
    replace a1, a2 with any previously folded constant value
    if op is binary and a1, a2 are both numeric literals:
        value = evaluate(op, a1, a2)
        remember result -> value // propagate forward
        DROP this instruction
    else: keep instruction (with substituted operands)
```

5.4 Common Sub-expression Elimination (CSE)

```
seen = {} // (op, arg1, arg2) -> earlier result
for each quad (op, a1, a2, res) in order:
    substitute a1, a2 with any earlier renaming
    key = (op, a1, a2)
    if key in seen:
        rename[res] = seen[key] // reuse earlier temp
```

```

    DROP this instruction
else:
    seen[key] = res; keep instruction

```

5.5 Dead Code Elimination (DCE)

```

live = { program outputs }    // e.g. {finalValue}
for each quad (op, a1, a2, res) taken in REVERSE order:
    if res in live:
        keep instruction
        add a1 (and a2) to live
    else:
        DROP instruction (its result is never used)

```

5.6 Driving the Optimizer to a Fixed Point

```

repeat up to N times:
    quads, c1 = constantFolding(quads)
    quads, c2 = commonSubexprElim(quads)
    quads, c3 = deadCodeElim(quads, liveOutputs)
until (c1 == false and c2 == false and c3 == false)

```

6. Implementation / Source Code

The module is split into two files: `compiler.py` (lexer, parser, AST, TAC generator, quadruple/triple builders, and the three optimization passes) and `run_demo.py` (the driver that runs the pipeline on the finance expression and on a synthetic program and prints every intermediate artifact). Both are reproduced below in full for completeness.

6.1 `compiler.py`

```

"""
Mini Intermediate-Code-Generation Module
=====
Pipeline: source text -> tokens -> AST -> Three Address Code (TAC)

```


-> Quadruples / Triples -> Optimized TAC/Quadruples

Author: Compiler Design Mini-Project

"""

import re

from dataclasses import dataclass, field

from typing import Optional, List, Tuple, Dict

1. LEXER

TOKEN_SPEC = [

("NUMBER", r"\d+(\.\d+)?"),

("IDENT", r"[A-Za-z_][A-Za-z0-9_]*"),

("POW", r"\^"),

("MUL", r"*"),

("DIV", r"/"),

("PLUS", r"\+"),

("MINUS", r"-"),

("LPAREN", r "("),

("RPAREN", r ")"),

("ASSIGN", r "="),

("SEMI", r ";"),

("SKIP", r "\t\n" +),

]

MASTER_RE = re.compile("|".join(f"(?P<{n}>{p})" for n, p in TOKEN_SPEC))

@dataclass

```

class Token:
    kind: str
    value: str

def tokenize(text: str) -> List[Token]:
    tokens = []
    pos = 0
    while pos < len(text):
        m = MASTER_RE.match(text, pos)
        if not m:
            raise SyntaxError(f"Unexpected character {text[pos]!r} at position {pos}")
        kind = m.lastgroup
        value = m.group()
        pos = m.end()
        if kind != "SKIP":
            tokens.append(Token(kind, value))
    tokens.append(Token("EOF", ""))
    return tokens

# -----
# 2. AST NODES
# -----

@dataclass
class Num:
    value: str

@dataclass
class Var:

```

```

name: str

@dataclass
class BinOp:
    op: str
    left: "object"
    right: "object"

@dataclass
class Assign:
    target: str
    expr: "object"

# -----
# 3. RECURSIVE-DESCENT PARSER
# assignment -> IDENT '=' expr
# expr      -> term (('+'|'-') term)*
# term      -> power (('*'|'/') power)*
# power     -> factor ('^' power)?    (right associative)
# factor    -> NUMBER | IDENT | '(' expr ')'
# -----

class Parser:
    def __init__(self, tokens: List[Token]):
        self.tokens = tokens
        self.pos = 0

    def peek(self) -> Token:
        return self.tokens[self.pos]

```

```

def advance(self) -> Token:
    tok = self.tokens[self.pos]
    self.pos += 1
    return tok

def expect(self, kind: str) -> Token:
    if self.peak().kind != kind:
        raise SyntaxError(f"Expected {kind} but got {self.peak().kind} ({self.peak().value!r})")
    return self.advance()

def parse_assignment(self) -> Assign:
    target = self.expect("IDENT").value
    self.expect("ASSIGN")
    expr = self.parse_expr()
    return Assign(target, expr)

def parse_program(self) -> List[Assign]:
    """A program is one or more assignment statements separated by ';'."""
    stmts = [self.parse_assignment()]
    while self.peak().kind == "SEMI":
        self.advance()
        if self.peak().kind == "EOF":
            break
        stmts.append(self.parse_assignment())
    self.expect("EOF")
    return stmts

def parse_expr(self):
    node = self.parse_term()
    while self.peak().kind in ("PLUS", "MINUS"):

```

```

        op = self.advance().value
        right = self.parse_term()
        node = BinOp(op, node, right)
    return node

def parse_term(self):
    node = self.parse_power()
    while self.peek().kind in ("MUL", "DIV"):
        op = self.advance().value
        right = self.parse_power()
        node = BinOp(op, node, right)
    return node

def parse_power(self):
    node = self.parse_factor()
    if self.peek().kind == "POW":
        self.advance()
        right = self.parse_power()      # right-associative
        node = BinOp("^", node, right)
    return node

def parse_factor(self):
    tok = self.peek()
    if tok.kind == "NUMBER":
        self.advance()
        return Num(tok.value)
    if tok.kind == "IDENT":
        self.advance()
        return Var(tok.value)
    if tok.kind == "LPAREN":
        self.advance()

```

```

        node = self.parse_expr()
        self.expect("RPAREN")
        return node
    if tok.kind == "MINUS":          # unary minus
        self.advance()
        node = self.parse_factor()
        return BinOp("-", Num("0"), node)
    raise SyntaxError(f"Unexpected token {tok.kind} ({tok.value!r})")

def parse(text: str) -> Assign:
    return Parser(tokenize(text)).parse_assignment()

def parse_program(text: str) -> List[Assign]:
    return Parser(tokenize(text)).parse_program()

# -----
# 4. THREE ADDRESS CODE (TAC) GENERATION
# Quad = (op, arg1, arg2, result)  arg2 is None for a plain copy/assign
# -----

class TACGenerator:
    def __init__(self):
        self.temp_count = 0
        self.quads: List[Tuple[str, str, Optional[str], str]] = []

    def new_temp(self) -> str:
        self.temp_count += 1
        return f"t{self.temp_count}"

```

```

def emit(self, op, arg1, arg2, result):
    self.quads.append((op, arg1, arg2, result))

def gen_expr(self, node) -> str:
    if isinstance(node, Num):
        return node.value
    if isinstance(node, Var):
        return node.name
    if isinstance(node, BinOp):
        left = self.gen_expr(node.left)
        right = self.gen_expr(node.right)
        temp = self.new_temp()
        self.emit(node.op, left, right, temp)
        return temp
    raise TypeError(f"Unknown AST node {node}")

def gen_assign(self, node: Assign):
    value = self.gen_expr(node.expr)
    self.emit("=", value, None, node.target)

def generate_tac(ast) -> List[Tuple[str, str, Optional[str], str]]:
    """Accepts either a single Assign (one statement) or a list of Assign
    (a multi-statement program) and returns the combined TAC quad list."""
    gen = TACGenerator()
    stmts = ast if isinstance(ast, list) else [ast]
    for stmt in stmts:
        gen.gen_assign(stmt)
    return gen.quads

```

```

def tac_to_lines(quads) -> List[str]:
    lines = []
    for op, a1, a2, res in quads:
        if op == "=":
            lines.append(f"{res} = {a1}")
        else:
            lines.append(f"{res} = {a1} {op} {a2}")
    return lines

# -----
# 5. QUADRUPLE / TRIPLE REPRESENTATIONS
# -----

def to_quadruple_table(quads):
    """List of dict rows: # | Op | Arg1 | Arg2 | Result"""
    rows = []
    for i, (op, a1, a2, res) in enumerate(quads):
        rows.append({"idx": i, "op": op, "arg1": a1, "arg2": a2 if a2 is not None else "-", "result":
res})
    return rows

def to_triple_table(quads):
    """
    Triples have no explicit result field; a temporary's *statement number*
    is used wherever that temporary is referenced.
    Returns list of dict rows: # | Op | Arg1 | Arg2
    """
    triples = []
    temp_to_index: Dict[str, int] = {}
    for i, (op, a1, a2, res) in enumerate(quads):

```



```

    r1 = f"({temp_to_index[a1]})" if a1 in temp_to_index else a1
    r2 = None
    if a2 is not None:
        r2 = f"({temp_to_index[a2]})" if a2 in temp_to_index else a2
    triples.append({"idx": i, "op": op, "arg1": r1, "arg2": r2 if r2 is not None else "-"})
    if res.startswith("t"):
        temp_to_index[res] = i
    return triples

```

```

# -----
# 6. OPTIMIZATION PASSES
# -----
NUM_RE = re.compile(r"^-?\d+(\.\d+)?\$")

```

```

def is_const(tok: str) -> bool:
    return bool(NUM_RE.match(tok))

```

```

def fold(op: str, a: float, b: float) -> float:
    if op == "+":
        return a + b
    if op == "-":
        return a - b
    if op == "*":
        return a * b
    if op == "/":
        return a / b
    if op == "^":
        return a ** b

```

```
raise ValueError(op)
```

```
def fmt_num(n: float) -> str:
```

```
    if isinstance(n, float) and n.is_integer():
```

```
        return str(int(n))
```

```
    return str(n)
```

```
def constant_folding(quads):
```

```
    """Fold ops whose both operands are numeric literals, then propagate
    the constant into later instructions that used the eliminated temp."""
```

```
    new_quads = []
```

```
    const_value: Dict[str, str] = {}
```

```
    def resolve(tok):
```

```
        return const_value.get(tok, tok)
```

```
    changed_any = False
```

```
    for op, a1, a2, res in quads:
```

```
        a1r, a2r = resolve(a1), (resolve(a2) if a2 is not None else None)
```

```
        if op != "=" and is_const(a1r) and a2r is not None and is_const(a2r):
```

```
            value = fold(op, float(a1r), float(a2r))
```

```
            const_value[res] = fmt_num(value)
```

```
            changed_any = True
```

```
            continue # instruction removed; folded into const_value table
```

```
        if op == "=" and is_const(a1r) and not res.startswith("t"):
```

```
            # constant assigned straight to a named/output variable - keep,
```

```
            # but rewrite operand with folded value
```

```
            new_quads.append((op, a1r, None, res))
```

```
        continue
```

```

    new_quads.append((op, a1r, a2r, res))
return new_quads, changed_any

```

```

def common_subexpression_elimination(quads):

```

```

    """Replace repeated (op, arg1, arg2) computations with the earlier temp."""

```

```

    seen: Dict[Tuple[str, str, Optional[str]], str] = {}

```

```

    rename: Dict[str, str] = {}

```

```

    new_quads = []

```

```

    changed_any = False

```

```

    def resolve(tok):

```

```

        return rename.get(tok, tok)

```

```

    for op, a1, a2, res in quads:

```

```

        a1r = resolve(a1)

```

```

        a2r = resolve(a2) if a2 is not None else None

```

```

        key = (op, a1r, a2r)

```

```

        if op != "=" and key in seen:

```

```

            rename[res] = seen[key]

```

```

            changed_any = True

```

```

            continue # duplicate computation dropped

```

```

        new_quads.append((op, a1r, a2r, res))

```

```

        if op != "=":

```

```

            seen[key] = res

```

```

    return new_quads, changed_any

```

```

def dead_code_elimination(quads, live_outputs):

```

```

    """Backward pass: keep an instruction only if its result is later used

```

```

    or is one of the program's externally visible outputs."""

```

```

live = set(live_outputs)
keep = [False] * len(quads)
for i in range(len(quads) - 1, -1, -1):
    op, a1, a2, res = quads[i]
    if res in live:
        keep[i] = True
        live.add(a1)
        if a2 is not None:
            live.add(a2)
new_quads = [q for q, k in zip(quads, keep) if k]
changed_any = (len(new_quads) != len(quads))
return new_quads, changed_any

```

```

def optimize(quads, live_outputs, max_passes=10):
    """Iteratively apply constant folding -> CSE -> DCE until a fixed point."""
    history = []
    current = list(quads)
    for _ in range(max_passes):
        current, c1 = constant_folding(current)
        current, c2 = common_subexpression_elimination(current)
        current, c3 = dead_code_elimination(current, live_outputs)
        history.append((c1, c2, c3))
        if not (c1 or c2 or c3):
            break
    return current, history

```

```

# -----
# 7. PRETTY PRINTERS
# -----

```

```
def print_quads(quads, title="Quadruples"):
    print(f"\n{title}")
    print(f"{'#':<4}{'Op':<6}{'Arg1':<14}{'Arg2':<14}{'Result':<10}")
    for i, (op, a1, a2, res) in enumerate(quads):
        print(f"{'i':<4}{'op':<6}{str(a1):<14}{str(a2) if a2 is not None else '-':<14}{res:<10}")

def print_triples(triples, title="Triples"):
    print(f"\n{title}")
    print(f"{'#':<4}{'Op':<6}{'Arg1':<14}{'Arg2':<14}")
    for row in triples:
        print(f"{'row['idx']':<4}{'row['op']':<6}{str(row['arg1']):<14}{str(row['arg2']):<14}")
```

6.2 run_demo.py

```
from compiler import (
    parse_program, generate_tac, tac_to_lines, to_quadruple_table, to_triple_table,
    optimize, print_quads, print_triples
)

def divider(title):
    print("\n" + "=" * 78)
    print(title)
    print("=" * 78)

def run_case(name, source, live_outputs):
    divider(f"CASE: {name}")
    print(f"Source expression:\n {source}\n")
```

```

ast = parse_program(source)
quads = generate_tac(ast)

print("---- Unoptimized Three Address Code (TAC) ----")
for line in tac_to_lines(quads):
    print(" ", line)

print_quads(quads, "---- Unoptimized Quadruples ----")
triples = to_triple_table(quads)
print_triples(triples, "---- Unoptimized Triples ----")

opt_quads, history = optimize(quads, live_outputs)

print("\n---- Optimized Three Address Code (TAC) ----")
for line in tac_to_lines(opt_quads):
    print(" ", line)

print_quads(opt_quads, "---- Optimized Quadruples ----")
opt_triples = to_triple_table(opt_quads)
print_triples(opt_triples, "---- Optimized Triples ----")

print("\n---- Optimization pass log (folding, CSE, DCE) ----")
for i, (c1, c2, c3) in enumerate(history, 1):
    print(f" Pass {i}: constant_folding_changed={c1} cse_changed={c2}
dce_changed={c3}")

unopt_count = len(quads)
opt_count = len(opt_quads)
reduction = unopt_count - opt_count
pct = (reduction / unopt_count * 100) if unopt_count else 0
print(f"\n---- Instruction Count Comparison ----")

```

```

print(f" Unoptimized instructions : {unopt_count}")
print(f" Optimized instructions  : {opt_count}")
print(f" Instructions removed    : {reduction} ({pct:.1f}% reduction)")
return unopt_count, opt_count

if __name__ == "__main__":
    # -----
    # Case 1: the actual domain expression from computational finance
    # -----
    finance_src = "finalValue=((principal*rate*time)/100)+(principal*(1+rate/100)^time)-fees"
    run_case("Compound Interest / Final Value Expression", finance_src,
live_outputs=["finalValue"])

    # -----
    # Case 2: synthetic 3-statement program engineered to exercise every
    # optimization technique in one place:
    # - "4*5"      -> constant folding
    # - "(a+b)*(a+b)" -> common subexpression elimination
    # - "deadVal=..." -> dead code elimination (never used afterwards)
    # -----
    synthetic_src = (
        "result=(a+b)*(a+b)+4*5;"
        "deadVal=a*b-7;"
        "finalOut=result-10"
    )
    run_case("Synthetic Program (demonstrates folding + CSE + DCE together)",
        synthetic_src, live_outputs=["finalOut"])

```

7. Test Cases and Expected / Actual Results

7.1 Test Case 1 — Financial Expression

Input:

```
finalValue = ((principal*rate*time)/100) + (principal*(1+rate/100)^time) - fees
```

Expected behaviour: the expression has no repeated sub-expressions, no operations between two literal constants, and every temporary contributes to the final result — so the optimizer is expected to leave the instruction count unchanged (0 reduction), which is itself a useful negative-result test confirming the optimizer never removes instructions that are actually required.

Actual unoptimized quadruples produced:

#	Op	Arg1	Arg2	Result
0	*	principal	rate	t1
1	*	t1	time	t2
2	/	t2	100	t3
3	/	rate	100	t4
4	+	1	t4	t5
5	^	t5	time	t6
6	*	principal	t6	t7
7	+	t3	t7	t8
8	-	t8	fees	t9
9	=	t9	-	finalValue

Actual optimized quadruples produced (identical — confirming the expected result):

Result: 10 unoptimized instructions -> 10 optimized instructions (0% reduction). Actual = Expected. PASS.

7.2 Test Case 2 — Synthetic Program (all three optimizations)

Input:

```
result = (a+b)*(a+b) + 4*5;
deadVal = a*b - 7;
```



```
finalOut = result - 10
```

Expected behaviour: '4*5' is a constant-constant multiplication (constant folding should reduce it to 20); '(a+b)' is computed twice (CSE should reuse the first temporary); 'deadVal' is assigned but never read by any later statement or by the live output finalOut (DCE should remove it entirely).

Actual unoptimized quadruples (11 instructions):

#	Op	Arg1	Arg2	Result
0	+	a	b	t1
1	+	a	b	t2
2	*	t1	t2	t3
3	*	4	5	t4
4	+	t3	t4	t5
5	=	t5	-	result
6	*	a	b	t6
7	-	t6	7	t7
8	=	t7	-	deadVal
9	-	result	10	t8
10	=	t8	-	finalOut

Actual optimized quadruples (6 instructions):

#	Op	Arg1	Arg2	Result
0	+	a	b	t1
1	*	t1	t1	t3
2	+	t3	20	t5
3	=	t5	-	result
4	-	result	10	t8
5	=	t8	-	finalOut

Observed transformations, matching every expectation:

- Constant folding: ' $4 * 5$ ' disappeared and was replaced in-line by the literal 20 in instruction 2.
- CSE: the duplicate ' $t2 = a + b$ ' was removed; instruction 1 now reuses $t1$ for both operands of the multiplication.
- DCE: the entire ' $deadVal = a*b - 7$ ' computation (3 instructions) was removed because $deadVal$ is never referenced again.

Result: 11 unoptimized instructions -> 6 optimized instructions (45.5% reduction). Actual = Expected. PASS.

8. Execution Screenshots / Output

The module was executed from the command line as `python3 run_demo.py`. Since this report was produced in a headless / non-graphical sandbox, an actual desktop screenshot could not be captured; the section below instead reproduces the complete, unedited console output of that run verbatim, which serves the same verification purpose.

```
=====
=====
CASE: Compound Interest / Final Value Expression
=====
=====
Source expression:
finalValue=((principal*rate*time)/100)+(principal*(1+rate/100)^time)-fees

---- Unoptimized Three Address Code (TAC) ----
t1 = principal * rate
t2 = t1 * time
t3 = t2 / 100
t4 = rate / 100
t5 = 1 + t4
t6 = t5 ^ time
```

```

t7 = principal * t6
t8 = t3 + t7
t9 = t8 - fees
finalValue = t9

```

---- Unoptimized Quadruples ----

#	Op	Arg1	Arg2	Result
0	*	principal	rate	t1
1	*	t1	time	t2
2	/	t2	100	t3
3	/	rate	100	t4
4	+	1	t4	t5
5	^	t5	time	t6
6	*	principal	t6	t7
7	+	t3	t7	t8
8	-	t8	fees	t9
9	=	t9	-	finalValue

---- Unoptimized Triples ----

#	Op	Arg1	Arg2
0	*	principal	rate
1	*	(0)	time
2	/	(1)	100
3	/	rate	100
4	+	1	(3)
5	^	(4)	time
6	*	principal	(5)
7	+	(2)	(6)
8	-	(7)	fees
9	=	(8)	-

---- Optimized Three Address Code (TAC) ----

```
t1 = principal * rate
t2 = t1 * time
t3 = t2 / 100
t4 = rate / 100
t5 = 1 + t4
t6 = t5 ^ time
t7 = principal * t6
t8 = t3 + t7
t9 = t8 - fees
finalValue = t9
```

---- Optimized Quadruples ----

#	Op	Arg1	Arg2	Result
0	*	principal	rate	t1
1	*	t1	time	t2
2	/	t2	100	t3
3	/	rate	100	t4
4	+	1	t4	t5
5	^	t5	time	t6
6	*	principal	t6	t7
7	+	t3	t7	t8
8	-	t8	fees	t9
9	=	t9	-	finalValue

---- Optimized Triples ----

#	Op	Arg1	Arg2
0	*	principal	rate
1	*	(0)	time
2	/	(1)	100
3	/	rate	100

```

4 + 1      (3)
5 ^ (4)    time
6 * principal (5)
7 + (2)    (6)
8 - (7)    fees
9 = (8)    -

```

---- Optimization pass log (folding, CSE, DCE) ----

Pass 1: constant_folding_changed=False cse_changed=False dce_changed=False

---- Instruction Count Comparison ----

Unoptimized instructions : 10

Optimized instructions : 10

Instructions removed : 0 (0.0% reduction)

```

=====
=====

```

CASE: Synthetic Program (demonstrates folding + CSE + DCE together)

```

=====
=====

```

Source expression:

result=(a+b)*(a+b)+4*5;deadVal=a*b-7;finalOut=result-10

---- Unoptimized Three Address Code (TAC) ----

t1 = a + b

t2 = a + b

t3 = t1 * t2

t4 = 4 * 5

t5 = t3 + t4

result = t5

t6 = a * b

```

t7 = t6 - 7
deadVal = t7
t8 = result - 10
finalOut = t8

```

---- Unoptimized Quadruples ----

#	Op	Arg1	Arg2	Result
0	+	a	b	t1
1	+	a	b	t2
2	*	t1	t2	t3
3	*	4	5	t4
4	+	t3	t4	t5
5	=	t5	-	result
6	*	a	b	t6
7	-	t6	7	t7
8	=	t7	-	deadVal
9	-	result	10	t8
10	=	t8	-	finalOut

---- Unoptimized Triples ----

#	Op	Arg1	Arg2
0	+	a	b
1	+	a	b
2	*	(0)	(1)
3	*	4	5
4	+	(2)	(3)
5	=	(4)	-
6	*	a	b
7	-	(6)	7
8	=	(7)	-
9	-	result	10

10 = (9) -

---- Optimized Three Address Code (TAC) ----

```
t1 = a + b
t3 = t1 * t1
t5 = t3 + 20
result = t5
t8 = result - 10
finalOut = t8
```

---- Optimized Quadruples ----

#	Op	Arg1	Arg2	Result
0	+	a	b	t1
1	*	t1	t1	t3
2	+	t3	20	t5
3	=	t5	-	result
4	-	result	10	t8
5	=	t8	-	finalOut

---- Optimized Triples ----

#	Op	Arg1	Arg2
0	+	a	b
1	*	(0)	(0)
2	+	(1)	20
3	=	(2)	-
4	-	result	10
5	=	(4)	-

---- Optimization pass log (folding, CSE, DCE) ----

```
Pass 1: constant_folding_changed=True cse_changed=True dce_changed=True
Pass 2: constant_folding_changed=False cse_changed=False dce_changed=False
```

---- Instruction Count Comparison ----

Unoptimized instructions : 11

Optimized instructions : 6

Instructions removed : 5 (45.5% reduction)

9. Analysis and Discussion

9.1 Instruction-count comparison

Test Case	Unoptimized Instr.	Optimized Instr.	Instructions Removed	% Reduction
Financial expression (Case 1)	10	10	0	0.0%
Synthetic program (Case 2)	11	6	5	45.5%

The two cases together tell a complete story. Case 1 shows that the optimizer is conservative and correct: because the finance formula genuinely has no redundant or dead computation and no constant-only sub-expressions, none of the three passes fire, and the optimizer correctly leaves all 10 instructions untouched rather than incorrectly 'simplifying' something that cannot be simplified. Case 2 shows the same optimizer, unmodified, achieving a 45.5% reduction in instruction count on a program engineered to contain exactly the three kinds of inefficiency the passes target.

9.2 Why the finance expression itself has few opportunities

Inspecting the AST for `finalValue` shows that every one of its nine binary operations is unique (no two operations share the same operator and pair of operands), every literal (100, 1) is combined with a variable rather than another literal, and every temporary feeds directly into the next operation on the way to `finalValue`. This is a realistic characteristic of many 'closed-form formula' expressions in scientific/financial DSLs — they are usually already written by a domain expert in a fairly minimal form. It also demonstrates an important, often overlooked point in compiler courses: optimizing compilers are not expected to always reduce instruction count; they are expected to reduce it whenever it is legally possible, and do nothing otherwise.

9.3 Quadruples vs. triples

The quadruple form's explicit result column made both constant folding and CSE simple substitutions ('wherever you see this result, use this value/temp instead'). Producing triples from the same TAC required an extra bookkeeping step (mapping each temporary name to the index of the triple that computed it), and — critically — every optimization pass in Section 6 operates on quadruples first, with triples regenerated afterwards, because deleting a quadruple only requires dropping one row, whereas deleting a triple would require renumbering every later positional reference. This matches the standard compiler-textbook trade-off: triples are more compact (no result field) but less convenient to optimize; quadruples use more memory but optimize easily.

9.4 Efficiency implications for the target machine

- Fewer TAC instructions generally translate to fewer register/temporary slots to allocate and fewer machine instructions after code generation, directly reducing code size.
- CSE additionally reduces redundant computation, so on real hardware Case 2's optimized version does strictly less arithmetic at run time (one addition instead of two, one multiplication instead of one wasted computation of $4*5$ at run time).
- DCE removes work that has no observable effect on the program's output at all, which is 'free' savings with zero risk to correctness.
- In the financial-formula case, because no instructions could be safely removed, the run-time cost is dictated entirely by the arithmetic complexity of the formula itself (2 additions/subtractions, 4 multiplications/divisions, and 1 power operation) — a useful reminder that optimization cannot substitute for an efficient algorithm/formula in the first place.

10. Conclusion

This project implemented a complete, working intermediate-code-generation module: a lexer and recursive-descent parser that builds an AST from arithmetic assignment statements; a TAC generator; converters to the quadruple and triple representations; and three classical optimization passes (constant folding, common sub-expression elimination, and dead code elimination) driven to a fixed point. Testing on the target financial formula confirmed the pipeline produces correct,

minimal TAC and that the optimizer safely makes no unnecessary changes when none are legally possible. Testing on a second, purpose-built program confirmed all three optimization techniques individually and in combination, cutting instruction count by 45.5%. The exercise reinforces the core compiler-design lesson that intermediate representations decouple front-end parsing from back-end code generation, and that peephole/local optimizations on that IR can meaningfully reduce the work a target machine must perform — when, and only when, the source program actually contains such redundancy.

11. Individual Contribution of Group Members

The table below is a template — replace the placeholder names/roles with your actual group members and the work each of you performed.

Name	Roll No.	Contribution
[Member 1 Name]	[Roll No.]	Lexer, parser, AST design (Sections 4, 5.1, 6.1)
[Member 2 Name]	[Roll No.]	TAC generation, quadruple/triple conversion (Sections 5.2, 6.1)
[Member 3 Name]	[Roll No.]	Optimization passes: folding, CSE, DCE (Sections 5.3–5.6)
[Member 4 Name]	[Roll No.]	Testing, results analysis, report compilation (Sections 7–10)

12. References

- A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers: Principles, Techniques, and Tools* (2nd ed.), Pearson, 2007 — chapters on Intermediate Code Generation and Code Optimization.
- K. C. Loudon, *Compiler Construction: Principles and Practice*, Cengage Learning, 1997.
- S. S. Muchnick, *Advanced Compiler Design and Implementation*, Morgan Kaufmann, 1997 — reference for constant folding, CSE, and dead-code elimination algorithms.
- Python Software Foundation, *Python 3 Language and re module documentation*, <https://docs.python.org/3/>.

- Project source code and console output produced by the authors for this assignment (Sections 6 and 8).